MongoDB(Basic)

Objective

By the end of this training, trainees will understand MongoDB fundamentals and be able to set up MongoDB locally, work with databases and collections, perform CRUD operations, write basic queries, use indexes and aggregation, and apply basic schema design and security practices.

Outline

1. Introduction to MongoDB
    a. What is MongoDB?
    b. MongoDB as a NoSQL Document Database
    c. MongoDB vs SQL Databases
    d. Key Characteristics of MongoDB
    e. MongoDB Use Cases
    f. When to Choose MongoDB vs When to Consider Another Database
2. MongoDB Architecture and Terminology
3. MongoDB Data Model
    a. Documents and BSON
    b. JSON vs BSON
    c. Document Design Basics
    d. Flexible Schema
    e. Nested Documents
    f. Arrays
    g. Basic Design Considerations
    h. `_id` and ObjectId
4. Databases and Collections
    a. Database Operations
    b. Collection Operations
5. CRUD Operations
    a. Create
    b. Read
        i. Projection
        ii. Sorting
        iii. Limiting Results
        iv. Skipping Results
    c. Update
    d. Delete
6. Query Operators
    a. Comparison Operators
    b. Logical Operators
    c. Element Operators
    d. Array Operators
    e. Querying Nested Documents
    f. Querying Arrays of Documents

7. Update Operators
    a. Field Update Operators
    b. Numeric Operators
    c. Array Update Operators
    d. Renaming Fields
8. Indexing Fundamentals
    a. Default `_id` Index
    b. Creating an Index
    c. Viewing Existing Indexes
    d. Seeing the Benefit of an Index
    e. Common Basic Index Types
        i. Single-Field Index
        ii. Compound Index
        iii. Unique Index
    f. Removing an Index
    g. Should We Create Indexes on Every Field?
9. Aggregation Fundamentals
    a. Aggregation Pipeline
    b. Calculating Average
    c. Combining Multiple Stages
10. MongoDB Relationships and Schema Design Basics
    a. One-to-One
    b. One-to-Many
    c. Many-to-Many
11. Basic Security and Best Practices
    a. Authentication and Authorization
    b. Do Not Expose MongoDB Publicly
    c. Protect Database Credentials
    d. Validate Application Data
    e. Creating Indexes Based on Usage
    f. Be Careful with Destructive Commands

Introduction

MongoDB is a document-oriented NoSQL database widely used for modern applications that require flexible data models and scalable data storage. This training introduces the core MongoDB concepts and provides practical examples to help developers understand how to store, query, update, and manage data effectively.

Introduction to MongoDB

What is MongoDB?

MongoDB is a **NoSQL, document-oriented database** used to store and retrieve application data. Unlike relational databases that organize data into tables and rows, MongoDB stores data as **documents**. A MongoDB document looks similar to a JSON object. Internally, MongoDB stores documents in BSON (Binary JSON) format, which supports additional data types beyond standard JSON.

```
{
  _id: ObjectId("65f123abc456789012345678"),
  name: "John",
  age: 28,
  department: "Engineering",
  skills: ["Java", "Spring Boot", "MongoDB"]
}
```

Here, the entire object represents one document. The document can contain simple fields, arrays, and nested objects. For example, an employee document can also contain an embedded address:

```
{
  name: "Alice",
  department: "Engineering",
  address: {
    city: "Pune",
    country: "India"
  }
}
```

This allows related data to be stored together when it makes sense for the application's access patterns.

Why do developers use MongoDB?

MongoDB is commonly used when applications need:

- A flexible data model
- Fast development and iteration
- Data containing nested objects or arrays
- High-volume read/write operations
- Horizontal scalability
- Easy integration with modern application stacks

For example, an e-commerce application could store an order as a single document:

```
{
  orderId: "ORD1001",
  customer: "John",
  items: [
    { product: "Laptop", quantity: 1 },
    { product: "Mouse", quantity: 2 }
  ],
  total: 75000
}
```

Instead of splitting this information across multiple tables, the application can retrieve the order and its items together as a document.

MongoDB as a NoSQL Document Database

MongoDB is a **NoSQL database**, meaning it does not use the traditional relational table-based model used by databases such as MySQL or PostgreSQL. MongoDB is specifically a **document database**. Data is stored as documents, which are grouped into collections.

The basic structure is:

```
Database
   └── Collection
         └── Document
               └── Fields
```

For example, an application might have:

```
ecommerce
   └── users
         ├── { name: "John", age: 28, ... }
         ├── { name: "Alice", age: 31, ... }
         └── { name: "David", age: 25, ... }
```

Why is this different from a relational database?

In a relational database, the same information might be represented using multiple tables:

Users:

id	name	email
1	John	john@example.com

Addresses:

user_id	city	country
1	Pune	India

MongoDB can represent the related information inside the same document when that design fits the application's requirements.

What does "NoSQL" mean here?

NoSQL does **not** mean "MongoDB cannot use SQL like queries" or "there is no structure." It generally refers to database systems that use data models other than the traditional relational table model. MongoDB provides structure through its **document model**, but documents in the same collection do not necessarily have to contain exactly the same fields.

For example:

```
// Document 1
{
  name: "John",
  department: "Engineering"
}

// Document 2
{
  name: "Alice",
  department: "HR",
  experience: 5
}
```

Both documents can exist in the same collection.

> 💡 **Key Takeaway**
> MongoDB uses a **document-based data model** where data is organized as **databases** → **collections** → **documents**, rather than **databases** → **tables** → **rows**. This model is particularly useful when application data naturally contains nested or variable structures.

MongoDB vs SQL Databases

For developers, the important difference is **how data is modeled and accessed**. MongoDB and relational databases can both handle common application requirements, but they approach data differently.

Relational Database	MongoDB
Database	Database
Table	Collection
Row	Document
Column	Field
Primary Key	_id

JOIN	$lookup / embedding
Fixed schema	Flexible document structure

Consider an employee record. In a relational database:

```
employees
--------------------------------
id | name  | department
1  | John  | Engineering
2  | Alice | HR
```

In MongoDB:

```
{
  _id: 1,
  name: "John",
  department: "Engineering"
}
```

Schema

Relational databases generally define the table structure before inserting data:

```
employees
--------------------------------
id        INTEGER
name      VARCHAR
age       INTEGER
```

MongoDB has a more flexible document structure. For example, documents in the same collection can have different fields:

```
// Document 1
{
  name: "John",
  department: "Engineering"
}

// Document 2
{
  name: "Alice",
  department: "HR",
  experience: 5
}
```

This flexibility can be useful when application requirements or data structures change frequently.

Relationships

Relational databases commonly use foreign keys and JOINs to work with related data. MongoDB provides two common approaches:

Embedding:

```
{
  name: "John",
  address: {
    city: "Pune",
    country: "India"
  }
}
```

Referencing:

```
{
  name: "John",
  addressId: ObjectId("...")
}
```

The choice depends on how the application reads and updates the data. We will cover **embedding vs referencing** in detail in the Schema Design section.

When does the difference matter?

As a developer, consider MongoDB when:

- Data is naturally represented as documents.
- The structure can change over time.
- Related data is frequently accessed together.
- The application requires horizontal scaling.

A relational database may be a better choice when:

- The data has many complex relationships.
- Strong relational constraints are central to the application.
- Complex transactional operations across many entities are common.

Key Characteristics of MongoDB

The following characteristics differentiate MongoDB from traditional relational databases and make it suitable for many modern applications:

Characteristic	Description
Document-oriented	Stores data as documents rather than rows and tables. Documents can contain nested objects and arrays.
Flexible schema	Documents within the same collection can have different fields, making it easier to evolve application data models.
Rich data model	Supports nested documents, arrays, embedded objects, and multiple BSON data types.
Scalable	Supports horizontal scaling through sharding, allowing data and workload to be distributed across multiple servers.
High availability	Replica sets maintain multiple copies of data and support automatic failover.
Rich querying	Provides query operators for filtering, sorting, updating, and working with nested data.
Aggregation	Provides an aggregation pipeline for operations such as grouping, filtering, transforming, and calculating data.
Developer-friendly	Documents closely resemble JSON objects, making MongoDB convenient to work with in modern application stacks.

For example, a MongoDB document can represent related data together:

```
{
  name: "John",
  department: "Engineering",
  skills: ["Java", "Spring Boot"],
  address: {
    city: "Pune",
    country: "India"
  }
}
```

This document-oriented structure allows developers to model application data in a way that closely matches the objects used by the application.

MongoDB Use Cases

MongoDB is commonly used when applications work with **flexible, nested, or rapidly changing data**. Some common use cases are:

E-commerce Applications

Product data can vary between products. MongoDB can store product-specific attributes in the same document.

```
{
  name: "Laptop",
  category: "Electronics",
  price: 75000,
  specifications: {
    ram: "16GB",
    storage: "512GB SSD"
  }
}
```

A different product can have a different set of specifications without requiring changes to a fixed table structure.

Content Management Systems

Articles, blogs, and other content can contain different structures such as tags, authors, images, and metadata.

```
{
  title: "Getting Started with MongoDB",
  author: "Alice",
  tags: ["MongoDB", "Database"],
  content: "...",
  metadata: {
    category: "Technology",
    language: "English"
  }
}
```

Real-Time Applications

Applications such as dashboards, notifications, messaging systems, and activity tracking can benefit from MongoDB's ability to handle frequent data updates.

For example, an activity document could look like:

```
{
  userId: 101,
  action: "LOGIN",
```

```
  timestamp: ISODate("2026-08-18T09:30:00Z"),
  device: "Web"
}
```

MongoDB also provides **Change Streams**, which can be used to react to database changes. Change Streams are covered in the Advanced section.

Catalog and Inventory Systems

MongoDB works well when items have different attributes.

For example, an electronics catalog might contain:

```
{
  name: "Laptop",
  price: 75000,
  specifications: {
    ram: "16GB",
    storage: "512GB"
  }
}
```

while a mobile phone document might contain:

```
{
  name: "Smartphone",
  price: 45000,
  specifications: {
    screenSize: "6.5 inch",
    battery: "5000mAh"
  }
}
```

Both can be stored in the same collection.

Logging and Event Data

Applications can generate large volumes of logs or events where the structure may vary.

```
{
  service: "payment-service",
  event: "PAYMENT_FAILED",
  timestamp: ISODate("2026-08-18T10:15:00Z"),
  details: {
    orderId: "ORD1001",
    reason: "TIMEOUT"
  }
}
```

MongoDB can be useful for storing and querying such event-oriented data.

When to Choose MongoDB vs When to Consider Another Database

MongoDB can be a good choice when	Consider another database when
Data is naturally represented as documents.	Data has highly complex relationships between many entities.
Data structure changes frequently.	A rigid relational schema and strong relational constraints are central to the application.
Related data is frequently accessed together.	The application relies heavily on complex JOINs across many entities.
The application needs horizontal scalability.	The workload is primarily relational and scales well with a traditional relational database.
The application handles large volumes of semi-structured data.	Complex multi-record transactions are the dominant operation.
Flexible schema is useful during application evolution.	Referential integrity and foreign-key relationships are critical requirements.
Use cases involve catalogs, content, events, logs, or similar document-oriented data.	Existing systems, reporting, or tooling are strongly built around relational databases.

MongoDB Architecture and Terminology

MongoDB organizes data in a simple hierarchy:

```
MongoDB Server
    │
    └── Database
          │
          └── Collection
                │
                └── Document
                      │
                      └── Field
```

For example:

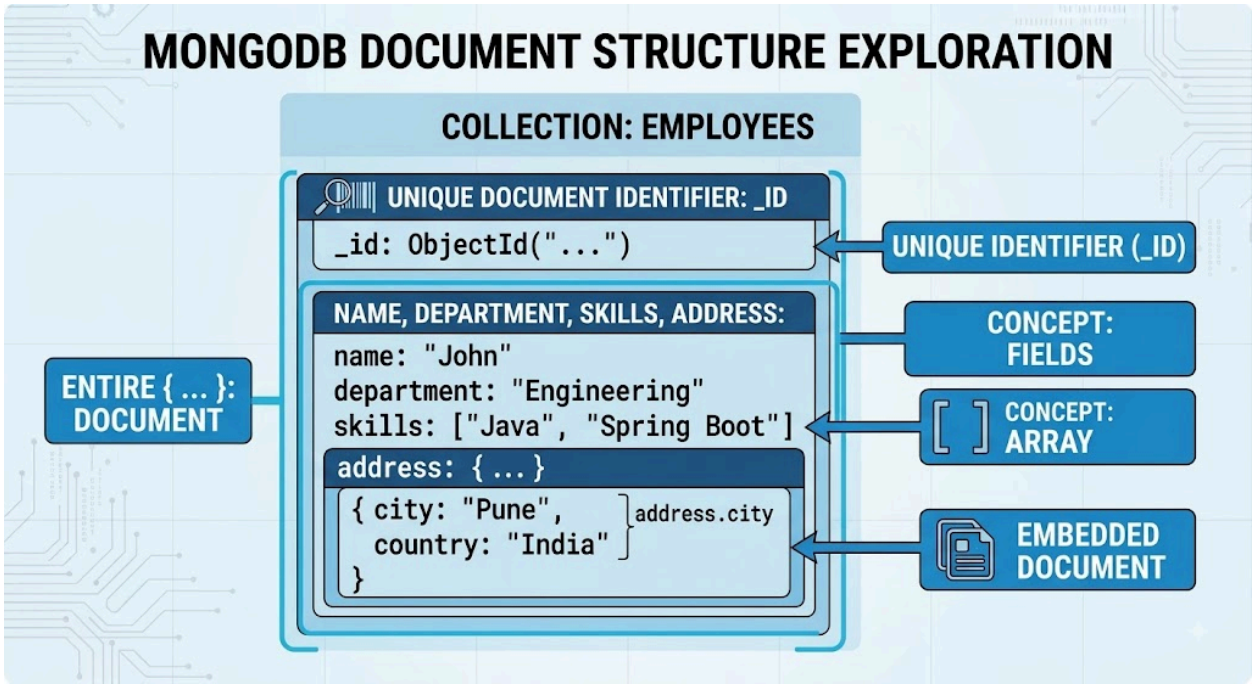
```
company
```

```
└── employees
      ├── { name: "John", department: "Engineering" }
      ├── { name: "Alice", department: "HR" }
      └── { name: "David", department: "Finance" }
```

Database: A container for collections.

- **Collection**: A group of related documents, similar to a table in a relational database.
- **Document**: An individual record stored as a BSON document.
- **Field**: A key-value pair within a document.
- **_id:** A unique identifier for each document.
- **BSON**: The binary representation used by MongoDB to store documents.
- **Embedded document**: A document nested inside another document.
- **Array**: A field that contains multiple values or documents.

For example:



This gives developers the basic mental model needed to work with MongoDB. The individual concepts such as BSON, _id, embedded documents, and arrays will be explored further when we start working with actual data.

MongoDB Data Model

Documents and BSON

A MongoDB document is stored internally as BSON. BSON stands for **Binary JSON**, but it is not simply a binary version of JSON. It is a binary-encoded serialization format based on JSON-like documents that

supports additional data types (such as ObjectId, Date, Int, Long, and Decimal) that are useful for database operations.

For example:

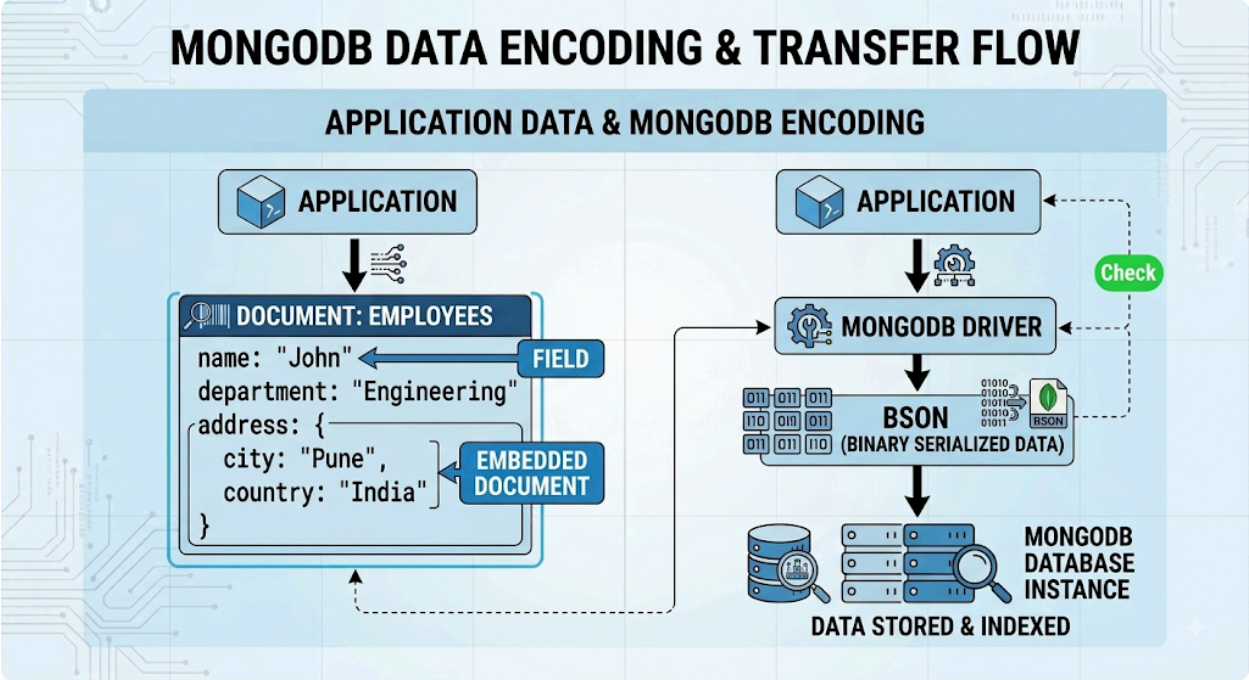
```
{
  name: "John",
  age: 28,
  active: true,
  joiningDate: ISODate("2026-01-15"),
  skills: ["Java", "Spring Boot"]
}
```

Some commonly used BSON types are:

BSON Type	Example	Common Usage
String	"John"	Names, descriptions, text
Int / Long	28	Numbers
Double	85.5	Decimal values
Decimal128	NumberDecimal("75000.50")	Precise decimal values such as prices, financial amounts, or calculations where floating-point rounding errors must be avoided
Boolean	true	Flags
Date	ISODate(...)	Timestamps and dates
Array	["Java", "MongoDB"]	Multiple values
Object	{ city: "Pune" }	Nested data
ObjectId	ObjectId("...")	Document identifiers
Null	null	Missing/empty values

JSON vs BSON

MongoDB documents look similar to JSON, but MongoDB stores them as BSON.



BSON provides additional data types, such as **ObjectId** and **Date**, that are important when working with MongoDB.

For example, a date can be stored as an actual BSON date rather than a string:

```
{
  name: "Alice",
  joiningDate: ISODate("2026-01-15")
}
```

This allows MongoDB to perform date based operations and comparisons correctly.

> 💡 **Key Takeaway**
> You can think of MongoDB documents as JSON-like objects, but MongoDB actually stores them using BSON, which provides additional data types and database-friendly behavior.

Document Design Basics

MongoDB's flexible document model allows developers to represent application data in a structure that closely matches how the application uses it. The goal is not simply to store data in any JSON-like

structure. **Documents should be designed around the application's access patterns and requirements.**

Flexible Schema

Documents within the same collection can contain different fields when required.

```
// Employee 1
{
 name: "John",
 department: "Engineering"
}

// Employee 2
{
 name: "Alice",
 department: "HR",
 experience: 5
}
```

This allows the data model to evolve without requiring every document to have exactly the same fields. However, **a flexible schema does not mean unstructured data**. Applications should still maintain consistency and apply validation where necessary.

Nested Documents

Related information can be embedded directly inside a document:

```
{
 name: "John",
 department: "Engineering",
 address: {
   city: "Pune",
   country: "India"
 }
}
```

This can be useful when the nested information is normally accessed together with the parent document.

Arrays

A document can contain arrays of values:

```
{
 name: "John",
 skills: ["Java", "Spring Boot", "MongoDB"]
}
```

Arrays can also contain documents:

```
{
 name: "John",
 projects: [
   {
     name: "Payment Service",
     status: "Active"
   },
   {
     name: "Order Service",
     status: "Completed"
   }
 ]
}
```

This allows related collections of data to be represented within a single document.

Basic Design Considerations

When designing MongoDB documents, developers should consider:

- How the application reads the data
- Which data is commonly accessed together
- Whether data should be embedded or referenced
- How frequently the data changes
- Avoiding unnecessarily large or continuously growing documents

Detailed **embedding vs referencing and advanced schema design patterns** will be covered later in the Schema Design section.

> 💡 **Key Takeaway**
> In MongoDB, design documents around **how the application uses the data**, rather than simply converting relational tables into MongoDB collections.

_id and ObjectId

Every MongoDB document has a unique _id field that identifies the document within its collection. If you don't provide an _id when inserting a document, MongoDB automatically generates one.

```
{
 name: "John",
 department: "Engineering"
}
```

MongoDB automatically adds an _id:

```
{
 _id: ObjectId("65f123abc456789012345678"),
 name: "John",
 department: "Engineering"
}
```

ObjectId

It's a *12-byte* BSON type that MongoDB uses as the default identifier for documents. It is designed to generate unique values across distributed systems without requiring coordination between servers, and it encodes a timestamp, making it roughly sortable by creation time.

Example: *ObjectId("65f123abc456789012345678")*

Applications can also provide their own _id value when appropriate:

```
{
 _id: "EMP1001",
 name: "John",
 department: "Engineering"
}
```

The _id value must be **unique within the collection**.

Why is _id important?

MongoDB uses _id to uniquely identify documents and provides an index on this field by default. This makes looking up a document by its identifier efficient.

For example:

```
db.employees.findOne({
 _id: ObjectId("65f123abc456789012345678")
})
```

Databases and Collections

In MongoDB, data is organized into **databases and collections**.

```
MongoDB Server
     │
     ├── company
     │      └── employees
     │            ├── Document 1
     │            └── Document 2
     │
     └── ecommerce
            └── products
                  ├── Document 1
                  └── Document 2
```

- A **database** contains one or more collections.
- A **collection** contains related documents.
- A collection is roughly comparable to a table in a relational database, although MongoDB's document model is different.

Database Operations

MongoDB does not require a separate command to create a database. A database is created when data is first stored in it.

To switch to or select a database:

```
use company
```

To see the current database:

```
db
```

To list available databases:

```
show dbs
```

> **Note:** A database with no stored data may not appear in `show dbs`.

To drop the current database:

```
db.dropDatabase()
```

Use this carefully, especially in a shared or production environment, because it permanently removes the database and its data.

Collection Operations

Collections store documents belonging to a particular purpose.

For example:

```
company
  ├── employees
  ├── departments
  └── projects
```

MongoDB can create a collection explicitly:

```
db.createCollection("employees")
```

You can list collections in the current database:

```
show collections
```

A collection can also be created **implicitly** when a document is inserted:

```
db.employees.insertOne({
 name: "John",
 department: "Engineering"
})
```

If **employees** does not already exist, MongoDB creates the collection automatically.

To remove a collection:

```
db.employees.drop()
```

CRUD Operations

CRUD stands for:

- **Create** – Add new documents
- **Read** – Retrieve documents
- **Update** – Modify existing documents
- **Delete** – Remove documents

These are the fundamental operations developers perform when working with MongoDB.

Create

Start by inserting three employees:

```
db.employees.insertMany([
  {
    name: "John",
    department: "Engineering",
    experience: 4,
    skills: ["Java", "Spring Boot"]
  },
  {
    name: "Alice",
    department: "HR",
    experience: 3,
    skills: ["Recruitment", "Communication"]
  },
  {
    name: "David",
    department: "Engineering",
    experience: 6,
    skills: ["Java", "MongoDB"]
  }
])
```

MongoDB generates an _id for each document automatically.

Now add another employee:

```
db.employees.insertOne({
  name: "Emma",
  department: "Finance",
  experience: 2,
  skills: ["Accounting", "Excel"]
})
```

Verify the documents:

```
db.employees.find()
```

At this point, the collection contains:

```
John   → Engineering → 4 years
Alice  → HR          → 3 years
David  → Engineering → 6 years
Emma   → Finance     → 2 years
```

Read

Retrieve all employees:

```
db.employees.find()
```

Filter employees by department:

```
db.employees.find({
  department: "Engineering"
})
```

```
// output
John
David
```

Find John:

```
db.employees.findOne({
  name: "John"
})
```

```
// Output
{
  name: "John",
  department: "Engineering",
  experience: 4,
  skills: ["Java", "Spring Boot"]
}
```

Projection

Return only the employee name and experience:

```
db.employees.find(
  { department: "Engineering" },
  { _id: 0, name: 1, experience: 1 }
)
```

```
[
  {
```

```
    name: "John",
    experience: 4
  },
  {
    name: "David",
    experience: 6
  }
]
```

Sorting

Sort employees by experience in descending order:

```
db.employees.find().sort({
  experience: -1
})
```

```
// Output
David → 6
John  → 4
Alice → 3
Emma  → 2
```

> `1` → Ascending
> `-1` → Descending

Limiting Results

Return only the first two documents:

```
db.employees.find().limit(2)
```

Skipping Results

Skip the first two documents:

```
db.employees.find().skip(2)
```

`skip()` and `limit()` are commonly used for pagination. For large datasets, more efficient pagination approaches may be preferred.

Update

Now we will modify the same employees created above.

`updateOne()`: Increase John's experience from 4 to 5:

```
db.employees.updateOne(
  { name: "John" },
  { $set: { experience: 5 } }
)
```

`$set` changes only the specified field.

Verify the change:

```
db.employees.findOne({
  name: "John"
})
```

```
// Output
{
  name: "John",
  department: "Engineering",
  experience: 5,
  skills: ["Java", "Spring Boot"]
}
```

`updateMany()`: Add an **`active`** field to all Engineering employees:

```
db.employees.updateMany(
  { department: "Engineering" },
  { $set: { active: true } }
)
```

Now both John and David will have: `active: true`

Verify:

```
db.employees.find(
  { department: "Engineering" },
  { _id: 0, name: 1, active: 1 }
)
```

```
// Output
[
  {
    name: "John",
    active: true
  },
  {
    name: "David",
    active: true
  }
]
```

`replaceOne()` replaces the entire document with the supplied document.

```
db.employees.replaceOne(
  { name: "Emma" },
  {
    name: "Emma",
    department: "Finance",
    experience: 3,
    skills: ["Accounting", "Excel"]
  }
)
```

The existing Emma document is replaced with the new document.
Any fields that exist in the original document but are **not included** in the replacement document will be **permanently removed**. For example, if Emma's original document had an `active: true` field but the replacement document does not include it, the `active` field will no longer exist after the replace operation. Use `$set` with `updateOne()` if you only want to modify specific fields without affecting the rest of the document.

> **Important:** Unlike `$set`, `replaceOne()` replaces the document rather than modifying selected fields.

Upsert

An upsert updates a matching document if it exists. If no matching document exists, MongoDB inserts a new document.

For example, suppose we want to update an employee named `Robert`

```
db.employees.updateOne(
  { name: "Robert" },
  {
    $set: {
      department: "Marketing",
      experience: 2
```

```
    }
  },
  { upsert: true }
)
```

Since `Robert` does not currently exist, MongoDB creates a new document.

Verify:

```
db.employees.findOne({
  name: "Robert"
})
```

```
// Output
{
  name: "Robert",
  department: "Marketing",
  experience: 2
}
```

Delete

We can now delete documents from the same collection.

`deleteOne():`
Delete Alice:

```
db.employees.deleteOne({
  name: "Alice"
})
```

Verify:

```
db.employees.find()
```

Alice is no longer present.

`deleteMany()`
Suppose we want to remove all employees from the Finance department:

```
db.employees.deleteMany({
  department: "Finance"
})
```

Verify:

```
db.employees.find()
```

Be Careful with Empty Filters
The following command matches **all documents** in the collection and deletes them.:

```
db.employees.deleteMany({})
```

Query Operators

MongoDB query operators allow developers to build conditions for retrieving documents.

For the examples below, continue using the same `employees` collection:

```
{
  name: "John",
  department: "Engineering",
  experience: 5,
  skills: ["Java", "Spring Boot"],
  active: true
}
```

Comparison Operators

Comparison operators are used to compare field values.

Operator	Meaning
`$eq`	Equal to
`$ne`	Not equal to
`$gt`	Greater than
`$gte`	Greater than or equal to
`$lt`	Less than
`$lte`	Less than or equal to
`$in`	Matches any value in an array
`$nin`	Does not match any value in an array

Examples:

Find employees with more than 4 years of experience:

```
db.employees.find({
  experience: { $gt: 4 }
})
```

Find employees with at least 5 years of experience:

```
db.employees.find({
  experience: { $gte: 5 }
})
```

Find employees with 3 or 5 years of experience:

```
db.employees.find({
  experience: { $in: [3, 5] }
})
```

Find employees who are not from HR:

```
db.employees.find({
  department: { $ne: "HR" }
})
```

Logical Operators

Logical operators allow multiple conditions to be combined.
Common operators:

- `$and`
- `$or`
- `$not`
- `$nor`

`$and`
Find Engineering employees with more than 4 years of experience:

```
db.employees.find({
  $and: [
    { department: "Engineering" },
    { experience: { $gt: 4 } }
```

```
  ]
})
```

MongoDB also allows this common shorthand:

```
db.employees.find({
  department: "Engineering",
  experience: { $gt: 4 }
})
```

`$or`
Find employees who are either in Engineering or HR:

```
db.employees.find({
  $or: [
    { department: "Engineering" },
    { department: "HR" }
  ]
})
```

Element Operators

Element operators are useful for checking whether a field exists or what type of value it contains.

`$exists`
Find documents where `active` exists:

```
db.employees.find({
  active: { $exists: true }
})
```

Find documents where `active` does not exist:

```
db.employees.find({
  active: { $exists: false }
})
```

`$type`
Find documents where `experience` is stored as an integer:

```
db.employees.find({
  experience: { $type: "int" }
})
```

Array Operators

MongoDB provides operators for querying array fields.
Suppose we have:

```
{
  name: "John",
  skills: ["Java", "Spring Boot", "MongoDB"]
}
```

$in
Find employees whose skills contain either Java or Python:

```
db.employees.find({
  skills: { $in: ["Java", "Python"] }
})
```

$all
Find employees who have both Java and MongoDB:

```
db.employees.find({
  skills: { $all: ["Java", "MongoDB"] }
})
```

$size
Find employees with exactly three skills:

```
db.employees.find({
  skills: { $size: 3 }
})
```

Querying Nested Documents

MongoDB allows queries on fields inside embedded documents.

For example:

```
{
  name: "John",
  address: {
    city: "Pune",
    country: "India"
  }
}
```

Find employees living in Pune:

```
db.employees.find({
  "address.city": "Pune"
})
```

The `.` notation is called **dot notation** and is used to access nested fields.

Querying Arrays of Documents

Consider:

```
{
  name: "John",
  projects: [
    {
      name: "Payment Service",
      status: "Active"
    },
    {
      name: "Order Service",
      status: "Completed"
    }
  ]
}
```

Find employees who have a project with an `Active` status:

```
db.employees.find({
  "projects.status": "Active"
})
```

For more complex conditions where multiple conditions must match the **same array element**, MongoDB provides `$elemMatch`.

```
db.employees.find({
  projects: {
    $elemMatch: {
      name: "Payment Service",
      status: "Active"
    }
  }
})
```

Update Operators

We already used `$set` in CRUD. This section focuses on the commonly used operators for modifying document fields without replacing the entire document.

For the examples, continue using the same `employees` collection.

Field Update Operators

`$set`
Sets or updates a field.

```
db.employees.updateOne(
  { name: "John" },
  { $set: { experience: 6 } }
)
```

You can also add a new field:

```
db.employees.updateOne(
  { name: "John" },
  { $set: { active: true } }
)
```

Only the specified fields are changed; the rest of the document remains unchanged.

`$unset`
Removes a field from a document.

```
db.employees.updateOne(
  { name: "John" },
  { $unset: { active: "" } }
)
```

The `active` field is removed from John's document.

Numeric Operators

`$inc`
Increases or decreases a numeric field.

Increase John's experience by 1:

```
db.employees.updateOne(
  { name: "John" },
  { $inc: { experience: 1 } }
)
```

If John's experience was `5`, it becomes `6`.

You can also decrease a value:

```
db.employees.updateOne(
  { name: "John" },
  { $inc: { experience: -1 } }
```

`$mul`
Multiplies a numeric field by a specified value.

For example, if an employee has a numeric `salary` field:

```
db.employees.updateOne(
  { name: "John" },
  { $mul: { salary: 1.10 } }
)
```

This increases John's salary by 10%.

Array Update Operators

Suppose John has:

```
{
  name: "John",
  skills: ["Java", "Spring Boot"]
}
```

`$push`
Adds an element to an array.

```
db.employees.updateOne(
  { name: "John" },
  { $push: { skills: "MongoDB" } }
)
```

Result:

```
skills: ["Java", "Spring Boot", "MongoDB"]
```

`$addToSet`

Adds an element only if it does not already exist in the array.

```
db.employees.updateOne(
  { name: "John" },
  { $addToSet: { skills: "MongoDB" } }
)
```

If `"MongoDB"` is already present, MongoDB does not add a duplicate.

Difference:

> `$push` → Adds the value even if it already exists
> `$addToSet` → Adds only if the value doesn't already exist

`$pull`

Removes matching values from an array.

```
db.employees.updateOne(
  { name: "John" },
  { $pull: { skills: "Spring Boot" } }
)
```

Result:

```
skills: ["Java", "MongoDB"]
```

Rename Fields

`$rename`

Changes the name of a field.

For example:

```
db.employees.updateOne(
  { name: "John" },
  { $rename: { "experience": "yearsOfExperience" } }
)
```

The field changes from:

```
experience: 6
```

to

```
yearsOfExperience: 6
```

Indexing Fundamentals

Indexes help MongoDB find data faster. Without an index, MongoDB may need to check many documents in a collection to find the required result.

For example:

```
db.employees.find({
  department: "Engineering"
})
```

If the `department` field is not indexed, MongoDB may scan the collection to find matching documents.

You can think of an index like the **index of a book**. Instead of reading every page, you use the index to quickly locate the required topic.

Default _id Index

MongoDB automatically creates an index on the `_id` field for every collection.

For example:

```
db.employees.find({
  _id: ObjectId("...")
})
```

MongoDB can use the `_id` index to quickly find that document. However, the `_id` index does not automatically help queries on other fields such as:

```
db.employees.find({
  department: "Engineering"
})
```

If `department` is frequently used for searching, we can create a separate index for it.

Creating an Index

Create an index on `department`:

```
db.employees.createIndex({
```

```
  department: 1
})
```

Here:

- 1 means ascending order.
- -1 means descending order.

MongoDB normally returns the name of the created index:

```
department_1
```

Now the collection has:

```
_id_          → Index on _id
department_1  → Index on department
```

Viewing Existing Indexes

To see the indexes created on a collection:

```
db.employees.getIndexes()
```

This shows the default `_id` index as well as any additional indexes created by the developer.

Seeing the Benefit of an Index

To demonstrate the difference, run the query before creating the index:

```
db.employees.find({
  department: "Engineering"
}).explain("executionStats")
```

Without an appropriate index, the query plan may show: `COLLSCAN`
It means MongoDB scanned the collection to find the matching documents.

Now create the index:

```
db.employees.createIndex({
  department: 1
})
```

Run the same query again:

```
db.employees.find({
```

```
  department: "Engineering"
}).explain("executionStats")
```

MongoDB may now show: `IXSCAN`
It means MongoDB used an index to locate the required data.

Key to remember:

> COLLSCAN → Collection scan
> IXSCAN → Index scan

Detailed query-plan analysis will be covered in the Advanced MongoDB section.

Common Basic Index Types

Single-Field Index

An index on one field:

```
db.employees.createIndex({
  department: 1
})
```

Useful when the application frequently searches using that field.

Compound Index

An index can contain more than one field:

```
db.employees.createIndex({
  department: 1,
  experience: -1
})
```

This is called a **compound index**.

For now we only need to know that compound indexes exist. How to choose the correct field order and optimize compound indexes will be covered in the Advanced section.

Unique Index

A unique index prevents duplicate values.

For example:

```
db.employees.createIndex(
  { email: 1 },
  { unique: true }
)
```

Now MongoDB will not allow two documents to have the same indexed `email` value.

Removing an Index

First check the index names:

```
db.employees.getIndexes()
```

Then remove an index:

```
db.employees.dropIndex("department_1")
```

The default `_id` index should normally be left as it is.

Should We Create Indexes on Every Field?

No. Indexes make reads faster, but they also:

- Use additional storage.
- Need to be updated when data changes.
- Can increase the cost of insert and update operations.

Therefore, developers should create indexes mainly on fields that are **frequently used in queries, sorting, or other important access patterns**.

Aggregation Fundamentals

Aggregation is used when we want to **process multiple documents and calculate or transform results**, rather than simply retrieve documents.

For example, with the same `employees` collection, we may want to find:

- Number of employees in each department
- Average experience by department
- Employees grouped by department
- Top employees based on experience

MongoDB performs these operations using an **aggregation pipeline**.

Aggregation Pipeline

An aggregation pipeline processes documents through a series of stages.

```
Documents
   ↓
$match
```

```
   ↓
$group
   ↓
$sort
   ↓
Result
```

Each stage performs one operation and passes its result to the next stage.

`$match`
It filters documents.

Find only Engineering employees:

```
db.employees.aggregate([
  {
    $match: {
      department: "Engineering"
    }
  }
])
```

This is similar to using:

```
db.employees.find({
  department: "Engineering"
})
```

The difference is that `$match` can be combined with other aggregation stages.

`$group`
It groups documents based on a field.

Count employees in each department:

```
db.employees.aggregate([
  {
    $group: {
      _id: "$department",
      totalEmployees: { $sum: 1 }
    }
  }
])
```

Result:

```
[
  {
    _id: "Engineering",
    totalEmployees: 2
  },
  {
    _id: "HR",
    totalEmployees: 1
  },
  {
    _id: "Finance",
    totalEmployees: 1
  }
]
```

Here:

- `_id: "$department"` groups employees by department.
- `$sum: 1` counts the number of documents in each group.

Calculating Average

We can calculate the average experience for each department:

```
db.employees.aggregate([
  {
    $group: {
      _id: "$department",
      averageExperience: {
        $avg: "$experience"
      }
    }
  }
])
```

For example:

```
Engineering → Average experience: 5.5
HR          → Average experience: 3
Finance     → Average experience: 3
```

`$project`

It controls which fields appear in the output and can also rename or calculate fields.

```
db.employees.aggregate([
  {
    $project: {
      _id: 0,
      name: 1,
```

```
      department: 1
    }
  }
])
```

```
// output
[
  {
    name: "John",
    department: "Engineering"
  },
  {
    name: "Alice",
    department: "HR"
  },
  {
    name: "David",
    department: "Engineering"
  },
  {
    name: "Emma",
    department: "Finance"
  }
]
```

Here:

- `name: 1` → include `name`
- `department: 1` → include `department`
- `_id: 0` → exclude `_id`

$sort

Sort aggregation results by experience:

```
db.employees.aggregate([
  {
    $sort: {
      experience: -1
    }
  }
])
```

Here:

- `1` → Ascending

- `-1` → Descending

`$limit`
Return only the first two results:

```
db.employees.aggregate([
  {
    $sort: {
      experience: -1
    }
  },
  {
    $limit: 2
  }
])
```

This can be used to get the two employees with the highest experience.

Combining Multiple Stages

The real benefit of aggregation comes from combining stages.

For example, find Engineering employees, sort them by experience, and return the top two:

```
db.employees.aggregate([
  {
    $match: {
      department: "Engineering"
    }
  },
  {
    $sort: {
      experience: -1
    }
  },
  {
    $limit: 2
  }
])
```

The flow is:

```
employees
   ↓
Engineering employees only
   ↓
Sort by experience
```

```
   ↓
Return top 2
```

MongoDB Relationships and Schema Design Basics

MongoDB allows related data to be modeled mainly in two ways:

- **Embedding**: store related data inside the same document.
- **Referencing**: store related data in separate documents and connect them using an identifier.

The right approach depends mainly on **how the application reads and updates the data**.

Relationships in MongoDB

MongoDB applications can have the same common relationship types found in other databases:

- **One-to-One** – One document is related to one other document.
- **One-to-Many** – One document is related to multiple documents.
- **Many-to-Many** – Multiple documents are related to multiple other documents.

One-to-One

Example: One employee has one address.

```
Employee
   |
   └── Address
```

This type of relationship is often a good candidate for **embedding** when the related data is usually accessed together.

```
{
  name: "John",
  address: {
    city: "Pune",
    country: "India"
  }
}
```

One-to-Many

Example: One department has multiple employees.

```
Department
   |
```

```
   ├── Employee 1
   ├── Employee 2
   └── Employee 3
```

Depending on the application requirement, the related data can be **embedded or referenced**.

Many-to-Many

Example: An employee can work on multiple projects, and a project can have multiple employees.

```
Employees                    Projects

Employee 1  ──────────────►  Project A
     │                          ▲
     └──────────────────────►  Project B
                                ▲
Employee 2  ────────────────────┘
```

Many-to-many relationships are commonly modeled using **references**, especially when both sets of data are managed independently.

> MongoDB does not enforce relationships through foreign keys like a relational database. Developers decide how relationships should be modeled using **embedding or referencing** based on how the application accesses and updates the data.

Basic Security and Best Practices

MongoDB should be configured so that only authorized users and applications can access the database. A few basic practices help protect both the database and its data.

Authentication and Authorization

Authentication verifies who is connecting to MongoDB, while **authorization** determines what that user is allowed to do.

For example, different users may have different permissions:

```
Developer
   │
   └── Read + Write access

Reporting User
   │
   └── Read-only access
```

Applications should follow the **least privilege principle**, a user should receive only the permissions required for their work.

Do Not Expose MongoDB Publicly

MongoDB should not be directly exposed to the internet unless there is a specific secured architecture requiring it.

A typical application flow should be:

```
User
  |
  v
Application / API
  |
  v
MongoDB
```

rather than:

```
User --------> MongoDB
```

Database access should be restricted to trusted applications, servers, and networks.

Common ways to protect database access include:

- **Firewall / Security Group Rules:** Configure firewalls or cloud security groups (e.g., AWS Security Groups, Azure NSGs) to allow connections to MongoDB only from trusted IP addresses or application servers.
- **Bind to specific IP addresses:** Configure MongoDB to listen only on internal/private network interfaces rather than `0.0.0.0` (all interfaces).
- **TLS/SSL Encryption:** Enable TLS to encrypt data in transit between the application and MongoDB, preventing data from being intercepted over the network.

Protect Database Credentials

Do not hard-code database usernames, passwords, or connection strings inside application code.

Avoid:

```
const uri = "mongodb://admin:password123@localhost:27017/company";
```

Instead, applications should obtain credentials from secure configuration such as environment variables or a secrets-management solution.

Validate Application Data

MongoDB provides a flexible schema, but applications should not accept uncontrolled or inconsistent data.

For example, if `experience` is expected to be numeric, the application should avoid storing inconsistent values such as:

```
{
  name: "John",
  experience: "five years"
}
```

when other documents contain:

```
{
  name: "Alice",
  experience: 5
}
```

Consistent data makes querying and application development easier.

Create Indexes Based on Usage

Do not create indexes on every field.

Create indexes based on fields that are frequently:

- Queried
- Filtered
- Sorted

As discussed in the **Indexing Fundamentals** section, unnecessary indexes consume storage and add overhead to write operations.

Be Careful with Destructive Commands

Commands such as:

```
db.employees.deleteMany({})

db.employees.drop()

db.dropDatabase()
```

can remove large amounts of data. Developers should always verify:

- The selected database
- The selected collection
- The query/filter condition

before executing destructive operations.

Gem

- https://gemini.google.com/u/3/gem/e61864ae4494

Reference material

- https://www.mongodb.com/docs/manual/
- https://www.mongodb.com/docs/manual/tutorial/getting-started/
- https://learn.mongodb.com/pages/mongodb-developer-learning-paths

Advanced reading material (good to have)

- https://www.mongodb.com/docs/manual/data-modeling/
- https://www.mongodb.com/docs/manual/crud/
- https://www.mongodb.com/docs/manual/indexes/
- https://www.mongodb.com/docs/manual/aggregation/